RELATÓRIO DE AUDITORIA DE SEGURANÇA

Segundo Cérebro AI (Second Brain)

Data: 17/10/2026 **Total:** 17 achados (1 alta, 4 média, 7 baixa, 5 informativa)

Escopo auditado: Repositório inteiro (86 arquivos rastreados, 13 commits): frontend Vite/JS (src/), Edge Functions Deno (supabase/functions), SQL e RLS (supabase/auth_rls.sql e migrations), workflow n8n, serviço media-fetcher (Python), Dockerfile, nginx.conf, GitHub Actions, service worker/PWA, testes e documentação.

Nota metodológica

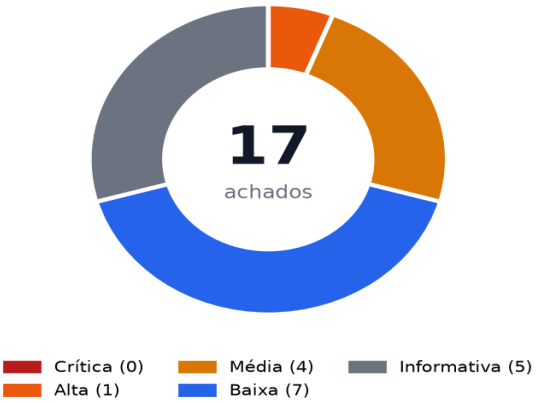
Esta auditoria mapeia cinco categorias de falha para a stack real do projeto. Só são reportados achados verificados no código; o que não pôde ser comprovado aparece como informativo.

Stack detectada	Frontend SPA/PWA em Vite com JavaScript puro (sem framework); backend Supabase (Postgres + pgvector + Auth + Edge Functions Deno); pipeline de ingestão em n8n; serviço auxiliar media-fetcher (Python, yt-dlp, ffmpeg); deploy por Dockerfile (nginx) via Coolify e GitHub Actions.
1. Banco sem tranca	O projeto é de usuário único. O isolamento é feito por RLS com a função public.is_owner() (auth.uid() igual ao UUID do dono), não por coluna de tenant. Foram verificadas todas as tabelas, políticas, funções RPC (security invoker) e grants nas migrations e em auth_rls.sql.
2. Permissão no navegador	Não existem papéis (isAdmin, canEdit, role) no frontend. O único gate do cliente é a sessão (main.js). Foi conferido se o backend impõe o mesmo: RLS, requireOwner nas Edge Functions e verificação de dono no n8n.
3. IDOR	Sem multiusuário, IDOR significa acesso a objeto por ID sem verificação de posse. Foram percorridos todos os handlers que recebem ID: brain-process (item_id), operações do supabaseClient.js (get/delete/update/merge/decisão de duplicata) e o webhook n8n.
4. Chaves expostas	Busca por padrões de segredo na árvore de trabalho e em todo o histórico git (13 commits, todas as branches), mais revisão manual de CI, Dockerfiles, workflow n8n, docs e bundle. O conteúdo de coolify_private_key.txt NÃO foi lido (bloqueio de segurança do ambiente); só se verificou se está ignorado e fora do histórico.
5. Inputs sem tratamento	Frontend: todos os usos de innerHTML, insertAdjacentHTML, href/src e eval em src/ (cerca de 45 ocorrências em 16 arquivos). Backend: entradas de URL que viram requisições (SSRF) em Edge Functions, n8n e media-fetcher, e entradas que viram texto exportado. Verificado por leitura de código; nenhum exploit foi executado contra ambientes reais.

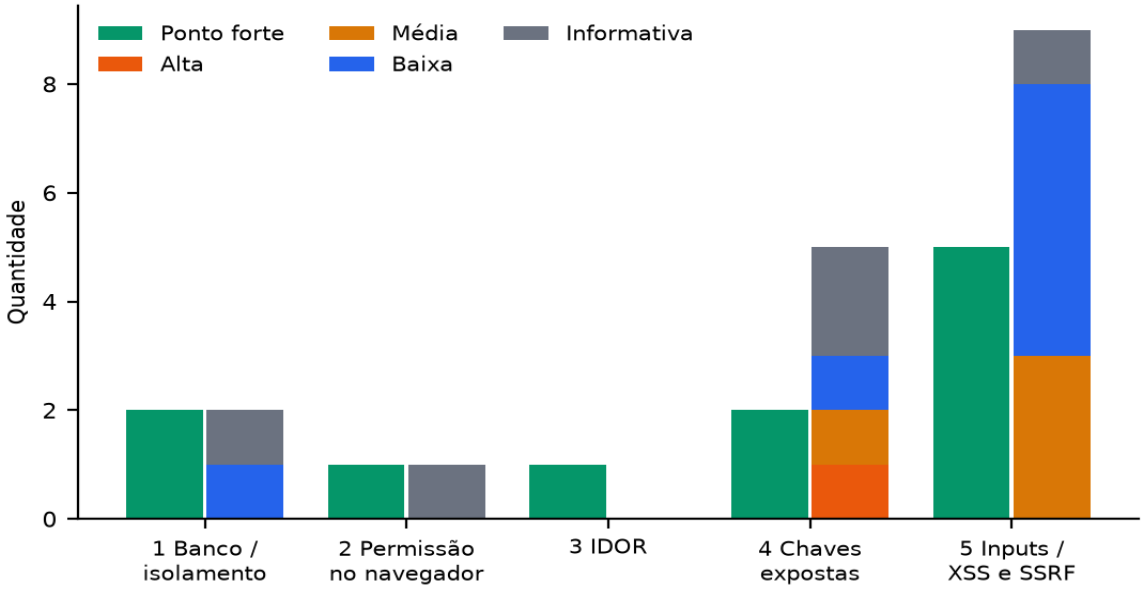
Resumo executivo

Não foram encontradas falhas críticas. Os dados do dono estão bem protegidos no banco (RLS em todas as tabelas, funções com security invoker) e o frontend escapa corretamente todo conteúdo dinâmico, sem XSS identificado. O risco concentra-se na **infraestrutura ao redor do app**: o token do Coolify trafega sem criptografia, há uma chave privada em pasta sincronizada com a nuvem, e dois pontos do pipeline (n8n e media-fetcher) buscam URLs sem barrar endereços internos (SSRF).

Severidade	Quantidade
CRÍTICA	0
ALTA	1
MÉDIA	4
BAIXA	7
INFORMATIVA	5
PONTO FORTE	11



Achados e pontos fortes por categoria



Pontos fortes e pontos fracos

Pontos fortes (com evidência)

Cat.	Ponto forte	Evidência
1	**RLS ativo em todas as tabelas do domínio, com política restrita ao dono**	brain_items e brain_chunks (auth_rls.sql:39-51), brain_atoms (migration 120000:168-174) e brain_duplicate_decisions (migration 130000:83-89): policy 'for all' com using e with check iguais a is_owner(), e 'revoke all ... from anon'.
1	**Funções RPC com security invoker e EXECUTE revogado de public/anon**	search_brain, brain_overview, brain_claim_item, brain_replace_item_knowledge, brain_item_links, brain_set_duplicate_decision, brain_merge_items, brain_set_practical_examples e owner_check: todas 'security invoker' (o RLS vale) e 'revoke execute ... from public, anon' (migrations 120000:509-519, 130000:238-246, 140000:26-27; auth_rls.sql:23-24).
2	**Backend impõe o acesso; o gate do navegador é só interface**	requireOwner valida o JWT e chama owner_check com cliente anon + token do usuário (runtime.ts:50-66) e é usado por brain-ask (index.ts:28) e brain-process (index.ts:18). O n8n chama owner_check antes de qualquer processamento (nós 'Credencial Supabase' e 'Verificar Dono') e repassa o token do usuário ao Supabase, então o RLS continua valendo.
3	**Nenhum acesso a objeto por ID sem verificação de posse**	Todos os handlers por ID (brain-process item_id validado como UUID em index.ts:10,43; getBrainItemById, deleteBrainItem, toggleItemFavorite, brain_merge_items, brain_set_duplicate_decision) passam pelo cliente com o token do usuário, portanto sob RLS is_owner. Não há service_role em nenhum arquivo.
4	**Nenhum segredo no código nem no histórico git**	Varredura de 13 commits (todas as branches) por chaves privadas, service_role, AIza, ghp_, sk-: sem resultados. GEMINI_API_KEY vem de secret da função, chaves do n8n são credenciais (id vazio no JSON), FETCHER_API_KEY vem do ambiente. coolify_private_key.txt está no .gitignore e nunca foi commitada.
4	**media-fetcher exige chave no start e compara em tempo constante**	server.py:307-308 encerra se FETCHER_API_KEY estiver vazia; server.py:285 usa hmac.compare_digest. Roda como usuário não-root (Dockerfile:7,17).
5	**Renderização segura: tudo é escapado antes de virar HTML**	markdown.js escapa o texto inteiro antes de aplicar Markdown (linhas 3-10, 38) e só monta links http/https com aspas escapadas. Nas ~45 ocorrências de innerHTML/insertAdjacentHTML de src/, todo dado dinâmico passa por escapeHtml ou textContent; os poucos valores sem escape são constantes do código (FeedFilters, CommandPalette). href vindo de dados é validado com /^https?:\/\// (ItemDetail.js:174, KnowledgeView.js:9). Há testes em tests/unit/markdown.test.mjs.
5	**Guard SSRF robusto e testado na Edge Function**	ssrf.ts bloqueia IPv4 privados, IPv6 (mapeado, NAT64, 6to4, Teredo), nomes internos, credenciais na URL e portas fora de 80/443 (linhas 32-96), revalida cada redirect, limita tamanho e tempo (116-154). Cobertura em tests/unit/ssrf.test.mjs.
5	**Defesa contra injeção de prompt e links inventados na resposta da IA**	ask.ts:230 instrui a nunca seguir instruções do contexto, e sanitizeAnswer (ask.ts:257-272) troca por '[link não verificado]' qualquer URL que não esteja no acervo recuperado.
5	**CORS com lista de origens (sem curinga) e limites de entrada nas funções**	runtime.ts:15-24 responde só origens permitidas; readJson limita o corpo a 200 KB (runtime.ts:37); brain-ask limita pergunta e histórico (index.ts:9-21).
5	**Dependências sem vulnerabilidades conhecidas**	npm audit: 0 vulnerabilidades (10 dependências de produção, 66 de desenvolvimento).

Pontos fracos (resumo)

ID	Sev.	Ponto fraco
4.1	ALTA	Token da API do Coolify enviado por HTTP sem criptografia
4.2	MÉDIA	Chave privada do Coolify guardada em pasta sincronizada com o OneDrive
5.1	MÉDIA	SSRF no n8n: o nó busca qualquer URL sem validar destino
5.2	MÉDIA	SSRF no media-fetcher: checagem de IP feita uma vez, sem cobrir redirects e re-resolução de DNS
5.4	MÉDIA	Sem Content-Security-Policy nem demais cabeçalhos de segurança
4.3	BAIXA	Workflow sem bloco permissions e com IP e UUID do servidor fixos no código
5.3	BAIXA	Guard SSRF da Edge Function: janela de DNS rebinding e falha de AAAA ignorada
5.5	BAIXA	Exportação .md injeta título, URL e conteúdo sem escapar
5.6	BAIXA	Parâmetros de URL e share-target pré-preenchem a captura sem origem confiável
5.7	BAIXA	Webhook do n8n sem limite de taxa antes da autenticação e CORS de produção com localhost
5.8	BAIXA	Cadeia de suprimentos e build: yt-dlp sem versão no start, npm install e .dockerignore curto
1.1	BAIXA	Esquema base e políticas RLS não são versionados nem testados contra um não-dono
4.4	INFORMATIVA	URL do projeto e anon key do Supabase fixas no bundle
4.5	INFORMATIVA	Artefatos de ferramenta e bytecode commitados
5.9	INFORMATIVA	Filtros PostgREST montados por string não removem aspas
1.2	INFORMATIVA	is_owner() e owner_check() sem search_path fixo; UUID do dono é placeholder editado à mão
2.1	INFORMATIVA	Login de fator único; bloqueio de novos cadastros, MFA e verify_jwt não verificáveis no repositório

Status da remediação

Situação após a execução do plano: o que foi corrigido no repositório (com teste) e o que ainda depende de ação manual em servidor, painel ou conta.

ID	Sev.	Status	O que foi feito / pendência manual
4.1	ALTA	CORRIGIDO	deploy.yml exige https://, usa secrets (COOLIFY_URL, COOLIFY_APP_UUID, COOLIFY_API_TOKEN) e permissions mínimas. **Pendente (você):** Colocar o Coolify em HTTPS e criar os secrets COOLIFY_URL e COOLIFY_APP_UUID no GitHub (token já rotacionado).
4.2	MÉDIA	PARCIAL	.dockerignore passou a excluir a chave e *.key/*.pem. **Pendente (você):** Mover coolify_private_key.txt para fora do OneDrive (e gerar novo par se houver dúvida de exposição).
5.1	MÉDIA	CORRIGIDO	Guard SSRF no nó 'Preparar Conteúdo' (IP/host/porta/credenciais, DoH, redirects revalidados, sem follow automático); 8 testes executam o código real do nó. **Pendente (você):** Reimportar o workflow no n8n.
5.2	MÉDIA	CORRIGIDO	Conexão fixada no IP validado, cada redirect revalidado, portas 80/443; 10 testes em media-fetcher/test_server.py (preflight falha fechado). **Pendente (você):** Bloquear egress do container para a rede interna e 169.254.169.254 (yt-dlp resolve DNS por conta própria).
5.4	MÉDIA	CORRIGIDO	CSP e demais cabeçalhos em todos os locations; script de tema inline virou arquivo externo; validado em navegador com os mesmos cabeçalhos (sem violações). **Pendente (você):** Confirmar com curl -I após o deploy (o Docker não rodou localmente).
1.1	BAIXA	CORRIGIDO	Migration 20261017110000 (esquema base + RLS + match_brain_chunks invoker) e tests/db/rls.test.mjs (dono, outro usuário, anon; mutação comprovada). **Pendente (você):** Aplicar auth_rls.sql novo e a migration em produção e conferir pg_policies/relrowsecurity.
4.3	BAIXA	CORRIGIDO	permissions: contents: read; IP e UUID saíram do repositório.
5.3	BAIXA	CORRIGIDO	createDohResolver: falha em A ou AAAA bloqueia a URL; testes novos. **Pendente (você):** Rebinding residual segue como risco aceito (supabase/README.md).
5.5	BAIXA	CORRIGIDO	obsidianExport.js: frontmatter com escape YAML, url só http(s), blocos executáveis (inclusive em citações/listas e em todos os campos) neutralizados; 7 testes.
5.6	BAIXA	CORRIGIDO	Só shared_* abrem a captura, com aviso de origem externa; service worker limita tipo e 10 MB. **Pendente (você):** Origem do POST do share-target não é validada para não quebrar o compartilhamento nativo.
5.7	BAIXA	PARCIAL	Nó 'Exigir Token' recusa requisições sem Bearer; CORS sem localhost no n8n e nas Edge Functions. **Pendente (você):** Rate limit por IP no proxy do Coolify (n8n não tem nativo).
5.8	BAIXA	CORRIGIDO	yt-dlp pinado (2026.8.19), sem pip no start; npm ci; .dockerignore completo.
1.2	INFORMATIVA	CORRIGIDO	Dono em public.brain_owner; is_owner security definer com search_path fixo; script aborta com UUID de zeros.
2.1	INFORMATIVA	PARCIAL	supabase/config.toml com verify_jwt pinado; README documenta cadastros e MFA. **Pendente (você):** Desativar cadastros e ativar MFA (TOTP) no painel do Supabase.
4.4	INFORMATIVA	ACEITO	Chave pública por design; proteção depende do RLS (versionado e testado) e de cadastros desativados. **Pendente (você):** Desativar cadastros no Supabase Auth.

ID	Sev.	Status	O que foi feito / pendência manual
4.5	INFORMATIVA	CORRIGIDO	.playwright-mcp e __pycache__ removidos do índice e ignorados no .gitignore.
5.9	INFORMATIVA	CORRIGIDO	sanitizeFilterTerm remove aspas duplas; testes novos.

Tabela de achados por categoria

1. Banco sem tranca (isolamento de dono/tenant)

Severidade	Arquivo:linha	Descrição
BAIXA	supabase/auth_rls.sql tests/helpers/db.mjs:39 supabase/README.md:27	**[1.1] Esquema base e políticas RLS não são versionados nem testados contra um não-dono** As tabelas brain_items e brain_chunks e a função match_brain_chunks foram criadas fora do repositório; só auth_rls.sql (rodado manualmente) as protege. Os testes simulam is_owner() como 'select true', então nenhum teste prova que um usuário comum é barrado. O estado real do RLS em produção não pode ser confirmado pelo código. **Evidência:** `create function public.is_owner() returns boolean language sql stable as 'select true';` **Condições de explorabilidade:** Só se o script manual não tiver sido executado, ou se match_brain_chunks em produção for SECURITY DEFINER (não verificável aqui). **Impacto:** Se o RLS faltar em alguma tabela, o anon key público dá leitura/escrita do acervo inteiro.
INFORMATIVA	supabase/auth_rls.sql:6-21	**[1.2] is_owner() e owner_check() sem search_path fixo; UUID do dono é placeholder editado à mão** As funções não definem search_path. Como não são SECURITY DEFINER, o risco é baixo. O UUID no script é 0000...; se não for trocado, o sistema falha fechado (ninguém acessa), o que é seguro, mas o valor real nunca está versionado. **Evidência:** `select auth.uid() = '00000000-0000-0000-0000-000000000000'::uuid` **Condições de explorabilidade:** Nenhuma exploração identificada. **Impacto:** Robustez e reprodutibilidade do ambiente.

2. Permissão definida no navegador

Severidade	Arquivo:linha	Descrição
INFORMATIVA	supabase/auth_rls.sql:3-4 ausência de supabase/config.toml	**[2.1] Login de fator único; bloqueio de novos cadastros, MFA e verify_jwt não verificáveis no repositório** O gate do cliente (main.js:520-540) é só de interface, e o servidor impõe o acesso (RLS, requireOwner, verificação no n8n). Mesmo assim, a única porta é e-mail e senha, sem MFA, e a desativação de cadastros é um passo manual descrito em comentário SQL. A configuração das funções (verify_jwt) não está pinada em config.toml. **Evidência:** `-- 2) Em Authentication > Sign In / Providers, desative "Allow new users to sign up".` **Condições de explorabilidade:** Senha fraca ou vazada do dono, ou cadastros abertos (o RLS ainda barraria os dados). **Impacto:** Tomada de conta por credencial vazada daria acesso total ao acervo.

3. IDOR

Sem achados. Verificado em todos os handlers aplicáveis: Nenhum acesso a objeto por ID sem verificação de posse.

4. Chaves expostas (hardcode)

Severidade	Arquivo:linha	Descrição
ALTA	.github/workflows/deploy.yml:15-19	**[4.1] Token da API do Coolify enviado por HTTP sem criptografia** O workflow chama http://116.203.95.162:8000/api/v1/deploy com o header Authorization: Bearer COOLIFY_API_TOKEN. Sem TLS, o token trafega em texto claro entre o runner do GitHub e o servidor, e pode ser capturado por qualquer ponto da rota. **Evidência:** `-H "Authorization: Bearer ${{ secrets.COOLIFY_API_TOKEN }}" \ | http://116.203.95.162:8000/api/v1/deploy)` **Condições de explorabilidade:** Requer um atacante com posição na rota de rede (ISP, nuvem, proxy). O token tem poder de disparar deploys no Coolify; o painel na porta 8000 também fica exposto à internet. **Impacto:** Com o token, é possível enfileirar deploys (e, conforme as permissões, ler/alterar recursos do Coolify). Isso permite substituir o app em produção por código malicioso.
MÉDIA	coolify_private_key.txt (raiz do projeto)	**[4.2] Chave privada do Coolify guardada em pasta sincronizada com o OneDrive** O arquivo está ignorado pelo git (.gitignore:25) e nunca foi commitado, mas fica em texto simples dentro de D:\OneDrive\..., ou seja, é replicado na nuvem e em todos os dispositivos sincronizados. O conteúdo não foi lido pela auditoria; o nome indica chave privada. **Evidência:** `$ git check-ignore -v coolify_private_key.txt | .gitignore:25:coolify_private_key.txt | $ git log --all --oneline -- coolify_private_key.txt (sem resultados)` **Condições de explorabilidade:** Exploração exige acesso à conta OneDrive, a outro dispositivo sincronizado ou a um backup. O .dockerignore também não exclui o arquivo (ver achado 5.8). **Impacto:** Se for a chave SSH/deploy do servidor, dá acesso ao host do Coolify, onde rodam n8n, media-fetcher e as variáveis de ambiente de produção.
BAIXA	.github/workflows/deploy.yml:1-12, 18-19	**[4.3] Workflow sem bloco permissions e com IP e UUID do servidor fixos no código** Não há 'permissions:' no workflow, então o GITHUB_TOKEN herda o padrão do repositório. O IP do servidor e o UUID da aplicação ficam em texto no repositório, facilitando reconhecimento. **Evidência:** `-d '{"uuid":"dog8k0ks8c4g8k0o48gswswc","force":false}' \ | http://116.203.95.162:8000/api/v1/deploy)` **Condições de explorabilidade:** Só vale se o repositório for tornado público ou compartilhado. O UUID sozinho não autoriza nada. **Impacto:** Facilita o mapeamento da infraestrutura; sem 'permissions', um passo comprometido teria mais poder do que o necessário.
INFORMATIVA	src/shared/constants/config.js:5-6	**[4.4] URL do projeto e anon key do Supabase fixas no bundle** A chave decodificada tem role 'anon' e a URL é pública por design no Supabase. Não é um segredo, mas toda a proteção depende de o RLS estar ativo no banco de produção. **Evidência:** `anonKey: 'eyJhbGciOiJIUzI1NiIs...' (payload: role = anon)` **Condições de explorabilidade:** Sem condição de exploração por si só. **Impacto:** Nenhum enquanto o RLS e o bloqueio de novos cadastros estiverem corretos (ver 1.1 e 2.1).
INFORMATIVA	.playwright-mcp/*, media-fetcher/__pycache__/server.cpython-313.pyc	**[4.5] Artefatos de ferramenta e bytecode commitados** Há um screenshot da tela de login, um snapshot YAML e um .pyc rastreados. Foram abertos e não contêm segredos nem dados reais, mas são lixo que pode vazar dados no futuro. **Evidência:** `git ls-files .playwright-mcp media-fetcher/__pycache__` **Condições de explorabilidade:** Nenhuma hoje. Risco apenas se artefatos futuros incluírem dados reais. **Impacto:** Higiene do repositório.

5. Inputs sem tratamento (XSS e injeção)

Severidade	Arquivo:linha	Descrição
MÉDIA	n8n/second_brain_workflow.json:343 (nó 'Preparar Conteúdo')	**[5.1] SSRF no n8n: o nó busca qualquer URL sem validar destino** Na estratégia 'web', o código faz httpGet(item.url) para qualquer host. Não há bloqueio de IPs privados, de localhost, de nomes internos do Docker, de portas ou de protocolos, e o cliente HTTP do n8n segue redirecionamentos por padrão. O texto obtido vai para o Gemini e é gravado em brain_items (raw_content/summary), ficando legível no app. A Edge Function brain-process tem um guard SSRF completo; o n8n não. **Evidência:** `const target = direct || item.url; | r = await httpGet(target, BROWSER_UA);` **Condições de explorabilidade:** Exige token do dono (o workflow valida owner_check antes). Vetores realistas: token roubado, ou o dono ser levado a salvar uma URL interna (ver 5.6). Verificado por leitura de código; não executado. **Impacto:** Leitura de serviços internos da rede do Coolify (ex.: http://media-fetcher:8080/health, painel do Coolify, o próprio n8n) e do endpoint de metadados da nuvem (169.254.169.254), com o conteúdo devolvido ao atacante pelo app.
MÉDIA	media-fetcher/server.py:46-57, 131-166, 243-248	**[5.2] SSRF no media-fetcher: checagem de IP feita uma vez, sem cobrir redirects e re-resolução de DNS** assert_public_url resolve o DNS só na entrada. Depois, download_direct usa urllib.request.urlopen, que segue redirects automaticamente, e yt-dlp resolve o DNS e segue redirects por conta própria. Um domínio público pode redirecionar (ou trocar de IP) para um endereço interno sem nova validação. Também não há restrição de porta. **Evidência:** `assert_public_url(url) # fetch_media, linha 244 | with urllib.request.urlopen(request, timeout=30) as resp: # linha 151` **Condições de explorabilidade:** O endpoint exige X-Api-Key e só é chamado pelo n8n após a validação do dono. A resposta devolvida é limitada a mídia (download_direct) e a metadados (yt-dlp), o que reduz a leitura de dados mas não o acesso cego a serviços internos. **Impacto:** Requisições a serviços internos e ao endpoint de metadados da nuvem (SSRF cego, com leitura parcial por título/descrição em extratores genéricos).
MÉDIA	nginx.conf:1-37 index.html:25-29	**[5.4] Sem Content-Security-Policy nem demais cabeçalhos de segurança** O nginx só define cache e gzip. Faltam CSP, X-Content-Type-Options, X-Frame-Options/frame-ancestors, Referrer-Policy, Permissions-Policy e HSTS. O app guarda a sessão do Supabase no localStorage (padrão do supabase-js) e carrega fonte do Google Fonts. Hoje não há XSS explorável (ver pontos fortes), mas qualquer falha futura roubaria o token do dono sem nenhuma barreira de defesa em profundidade. **Evidência:** `location / { try_files $uri $uri/ /index.html; } # nenhum add_header de segurança` **Condições de explorabilidade:** Defesa em profundidade: só importa se surgir um XSS ou for tentado clickjacking. **Impacto:** Clickjacking possível (sem frame-ancestors) e nenhuma contenção se houver XSS: roubo da sessão e acesso total ao acervo.
BAIXA	supabase/functions/_shared/runtime.ts:78-92 ssrf.ts:80-89	**[5.3] Guard SSRF da Edge Function: janela de DNS rebinding e falha de AAAA ignorada** O DNS é resolvido por DNS-over-HTTPS (Cloudflare), mas o fetch final usa o resolvedor do runtime, então há janela de rebinding (risco já aceito em supabase/README.md:76). Além disso, uma falha na consulta AAAA é engolida e a checagem passa a considerar só IPv4, embora o fetch possa conectar por IPv6. **Evidência:** `const [v4, v6] = await Promise.all([query('A'), | query('AAAA').catch(() => [] as string[])]);` **Condições de explorabilidade:** Exige controle de um domínio público e de seu DNS, além de token do dono. Mitigado por portas 80/443, 1 nível de profundidade e redirects revalidados. **Impacto:** Possível acesso a endereço interno em cenário de rebinding ou de AAAA que aponte para IPv6 privado.

Severidade	Arquivo:linha	Descrição
BAIXA	src/shared/utils/formatters.js:75-107	**[5.5] Exportação .md injeta título, URL e conteúdo sem escapar** O frontmatter só escapa aspas duplas no título; quebras de linha e o campo url não são tratados, e raw_content (texto vindo de páginas externas via IA) é gravado cru no arquivo. Um conteúdo malicioso pode injetar chaves YAML ou blocos de código executáveis por plugins do Obsidian (ex.: dataviewjs com JS habilitado). **Evidência:** `title: "${item.title.replace(/"/g, '\\"')}" \| url: "${item.source_url || ''}" | ${item.raw_content || 'Sem transcrição disponível.'}` **Condições de explorabilidade:** O dono precisa baixar o .md e abri-lo no Obsidian com um plugin que execute código do arquivo. Ataque indireto (conteúdo de terceiros capturado). **Impacto:** Execução de código no Obsidian do dono ou adulteração de metadados da nota exportada.
BAIXA	src/main.js:482-499 public/sw.js:35-68 public/manifest.json:24-49	**[5.6] Parâmetros de URL e share-target pré-preenchem a captura sem origem confiável** Qualquer link ?url=, ?text= ou ?shared_url= (e um POST cross-site para /share-target, tratado pelo service worker) abre a janela de captura pré-preenchida com URL, texto ou arquivo escolhidos pelo atacante. Nada é enviado automaticamente, o dono ainda precisa clicar em Salvar, mas a janela não indica a origem do conteúdo. **Evidência:** `const sharedUrl = urlParams.get('shared_url') || urlParams.get('url');` **Condições de explorabilidade:** Exige engenharia social: o dono abrir o link e confirmar o salvamento. Combina com o SSRF do n8n (5.1) para atingir URLs internas. **Impacto:** Captura induzida de URL interna ou de conteúdo malicioso no acervo.
BAIXA	n8n/second_brain_workflow.json:4-22 supabase/functions/_shared/runtime.ts:13	**[5.7] Webhook do n8n sem limite de taxa antes da autenticação e CORS de produção com localhost** O webhook é público e a validação de dono só ocorre depois de receber e parsear o corpo (até o limite do n8n). Cada chamada não autenticada dispara uma RPC ao Supabase. As origens permitidas incluem http://localhost:5173 (n8n) e localhost:5173/4173 (Edge Functions) em produção. CORS não é autenticação; o risco é de abuso e superfície extra. **Evidência:** `"allowedOrigins": "https://2cerebro.scaladigital.com.br,http://localhost:5173" | const DEFAULT_ORIGINS = ['https://2cerebro...', 'http://localhost:5173', 'http://localhost:4173'];` **Condições de explorabilidade:** Qualquer pessoa que descubra a URL do webhook (ela é pública no bundle). **Impacto:** Consumo de recursos (negação de serviço leve) e origens de desenvolvimento aceitas em produção.
BAIXA	media-fetcher/Dockerfile:22 Dockerfile:8,15 .dockerignore:1-6	**[5.8] Cadeia de suprimentos e build: yt-dlp sem versão no start, npm install e .dockerignore curto** O media-fetcher instala a última versão do yt-dlp a cada start (código novo não revisado rodando com acesso à rede interna). O Dockerfile principal usa npm install em vez de npm ci e copia o contexto inteiro; o .dockerignore não exclui .env*, *.key, coolify_private_key.txt, tests nem .playwright-mcp. **Evidência:** `CMD ["sh", "-c", "pip install --quiet -U 'yt-dlp[default]' || true; exec python /app/server.py"] | RUN npm install` **Condições de explorabilidade:** Comprometimento do pacote yt-dlp no PyPI, ou build local de imagem com arquivos sensíveis na pasta (o Coolify constrói a partir do git, onde o arquivo não existe). **Impacto:** Execução de código não auditado no host de mídia; segredos locais podem entrar em camadas de imagem construídas na máquina do dev.

Severidade	Arquivo:linha	Descrição
INFORMATIVA	src/shared/services/supabaseClient.js:44-51, 226-233	**[5.9] Filtros PostgREST montados por string não removem aspas** A busca remove vírgula, parênteses, %, * e barra, mas não aspas duplas. Uma aspa solta gera filtro malformado (erro 400). Como o RLS limita tudo ao dono, não há acesso a dados de terceiros: o efeito é só erro de busca. **Evidência:** `.replace(/[,()%*\\]/g, ' ').replace(/\s+/g, ' ').trim()` **Condições de explorabilidade:** Apenas o próprio dono digita a busca. **Impacto:** Erro de busca com aspas; sem impacto de segurança.

Recomendações priorizadas

P1 — Imediato (hoje)

Issue	Sev.	Ação
#1	ALTA	Migrar o deploy para HTTPS, rotacionar o token do Coolify, mover IP e UUID para secrets e limitar permissions.
#2	MÉDIA	Tirar coolify_private_key.txt do OneDrive; revogar e gerar novo par de chaves se houver qualquer dúvida de exposição.

P2 — Próximos dias

Issue	Sev.	Ação
#3	MÉDIA	Aplicar validação SSRF (IP privado, redirects, portas) no nó 'Preparar Conteúdo' do n8n.
#4	MÉDIA	Revalidar cada redirect no media-fetcher e restringir o egress do container.
#5	MÉDIA	Adicionar CSP e cabeçalhos de segurança no nginx.

P3 — Próximas semanas

Issue	Sev.	Ação
#11	BAIXA	Versionar o RLS, testar dono x não-dono, ativar MFA e confirmar cadastros desativados.
#7	BAIXA	Escapar a exportação .md e os filtros de busca.
#8	BAIXA	Sinalizar origem externa nos pré-preenchimentos de captura.
#9	BAIXA	Rate limit no webhook e remover localhost do CORS de produção.
#10	BAIXA	Pinar yt-dlp, npm ci, completar .dockerignore e limpar artefatos rastreados.

P4 — Backlog

Issue	Sev.	Ação
#6	BAIXA	Endurecer o guard SSRF das Edge Functions (AAAA e rebinding).

ISSUES PARA O GITHUB

11 issues agrupam os 16 achados acionáveis (achados triviais relacionados foram unidos). Copie o texto entre os marcadores de cada issue.

Issue 1 — [Segurança] Token do Coolify trafega por HTTP no deploy.yml

````
--- ISSUE 1 ---
Título: [Segurança] Token do Coolify trafega por HTTP no deploy.yml
Labels sugeridas: security, severity:alta

## Problema
O workflow de deploy envia COOLIFY_API_TOKEN por http:// (sem TLS) para o IP do servidor. Qualquer
ponto da rota pode ler o token e usá-lo para disparar deploys e alterar recursos do Coolify. O
workflow também não define 'permissions:' e deixa IP e UUID do servidor no repositório.

## Evidência
Arquivo: .github/workflows/deploy.yml:15-19
```
-H "Authorization: Bearer ${{ secrets.COOLIFY_API_TOKEN }}" \
http://116.203.95.162:8000/api/v1/deploy)
```

## Impacto
Tomada de deploy: um atacante na rota publica código malicioso no app em produção.

## Sugestão de correção
- Colocar o Coolify atrás de HTTPS (domínio + certificado) e trocar a URL para https://.
- Rotacionar o COOLIFY_API_TOKEN (considerar o atual exposto) com escopo mínimo.
- Mover URL e UUID para secrets/variables do repositório (COOLIFY_URL, COOLIFY_APP_UUID).
- Adicionar 'permissions: contents: read' ao workflow.
- Fechar a porta 8000 ao público, ou restringir por firewall aos IPs necessários.

## Critérios de aceite
- [ ] O workflow usa somente https:// e o deploy automático continua funcionando.
- [ ] O token antigo foi revogado e o novo está só em secrets.
- [ ] IP e UUID não aparecem mais no repositório.
- [ ] O workflow declara 'permissions' mínimas.

--- FIM ISSUE 1 ---
````

Issue 2 — [Segurança] Chave privada do Coolify em pasta sincronizada com o OneDrive

```
--- ISSUE 2 ---
Título: [Segurança] Chave privada do Coolify em pasta sincronizada com o OneDrive
Labels sugeridas: security, severity:media

## Problema
O arquivo coolify_private_key.txt está na raiz do projeto, dentro de uma pasta replicada pelo
OneDrive. Está ignorado pelo git e nunca foi commitado (verificado), mas fica em texto simples na
nuvem e em todos os dispositivos sincronizados. O conteúdo não foi lido pela auditoria.

## Evidência
Arquivo: coolify_private_key.txt (raiz); .gitignore:25
```
$ git check-ignore -v coolify_private_key.txt
.gitignore:25:coolify_private_key.txt
```

## Impacto
Quem acessar a conta OneDrive ou um dispositivo sincronizado obtém acesso ao servidor do Coolify.

## Sugestão de correção
- Mover a chave para fora do OneDrive (ex.: ~/.ssh com permissão restrita) ou para um gerenciador de
  segredos.
- Se a conta OneDrive for compartilhada ou houver dúvida, gerar um novo par de chaves e revogar a
  antiga no servidor.
- Excluir a cópia do OneDrive, inclusive da lixeira e do histórico de versões.
- Adicionar o arquivo ao .dockerignore (ver issue de build).

## Critérios de aceite
- [ ] Nenhuma cópia da chave existe em pastas sincronizadas.
- [ ] A chave antiga foi revogada se houve exposição.
- [ ] O deploy continua funcionando com a nova chave.

--- FIM ISSUE 2 ---
```

Issue 3 — [Segurança] SSRF no workflow n8n: nó 'Preparar Conteúdo' busca qualquer URL

```
--- ISSUE 3 ---
Título: [Segurança] SSRF no workflow n8n: nó 'Preparar Conteúdo' busca qualquer URL
Labels sugeridas: security, severity:media

## Problema
Na estratégia 'web', o nó executa httpGet(item.url) sem validar o destino. Não há bloqueio de IPs
privados, localhost, nomes internos do Docker, portas ou protocolos, e redirecionamentos são
seguidos. O conteúdo retornado é enviado ao Gemini e gravado em brain_items, ficando legível no app.
A Edge Function brain-process já tem um guard completo (supabase/functions/_shared/ssrf.ts); o n8n
não.

## Evidência
Arquivo: n8n/second_brain_workflow.json:343 (nó Preparar Conteúdo)
```
const target = direct || item.url;
r = await httpGet(target, BROWSER_UA);
```

## Impacto
Leitura de serviços internos (media-fetcher, painel do Coolify, n8n) e do endpoint de metadados da
nuvem (169.254.169.254) por quem tiver o token do dono ou induzir o dono a salvar uma URL interna.

## Sugestão de correção
- Validar a URL antes de buscar: só http/https, sem credenciais, portas 80/443, hostname com ponto,
  e bloquear IPs privados/loopback/link-local/metadata (v4 e v6).
- Resolver o DNS e validar o IP; revalidar a cada redirecionamento (desabilitar o follow automático
  e tratar manualmente).
- Alternativa: remover a busca web do n8n e delegar a captura a uma função com o guard de ssrf.ts.
- Manter a rede do n8n sem rota para o painel do Coolify e para o endpoint de metadados.

## Critérios de aceite
- [ ] URLs como http://127.0.0.1, http://media-fetcher:8080, http://169.254.169.254 e http://[::1]
  são rejeitadas com erro claro.
- [ ] Um redirect de um domínio público para IP interno é rejeitado.
- [ ] Há teste (ou execução manual documentada) cobrindo esses casos.
- [ ] Capturas de sites públicos continuam funcionando.

--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] media-fetcher: SSRF por redirect e re-resolução de DNS

```
--- ISSUE 4 ---
Título: [Segurança] media-fetcher: SSRF por redirect e re-resolução de DNS
Labels sugeridas: security, severity:media

## Problema
assert_public_url valida o DNS apenas na entrada de fetch_media. download_direct usa
urllib.request.urlopen, que segue redirects sem revalidar, e yt-dlp resolve DNS e segue redirects
por conta própria. Um domínio público pode redirecionar ou mudar de IP para um endereço interno.
Também não há restrição de porta.

## Evidência
Arquivo: media-fetcher/server.py:151, 244
```
assert_public_url(url)   # fetch_media
with urllib.request.urlopen(request, timeout=30) as resp:
```

## Impacto
Requisições a serviços internos e ao endpoint de metadados da nuvem (SSRF cego, leitura parcial via
metadados do yt-dlp).

## Sugestão de correção
- No download direto, usar um opener sem redirect automático e revalidar (assert_public_url) a cada
  hop.
- Resolver o host uma vez, validar o IP e conectar nesse IP (evita rebinding).
- Restringir portas a 80/443 e proibir esquemas diferentes de http/https também após redirects.
- Para yt-dlp: executar em rede sem acesso à rede interna (egress filtering) e bloquear
  169.254.169.254 no firewall do container.

## Critérios de aceite
- [ ] Redirect de URL pública para http://169.254.169.254 ou 127.0.0.1 é recusado.
- [ ] Um host que muda de IP entre a checagem e a conexão não alcança IP interno.
- [ ] O container não alcança a rede interna do Coolify nem o endpoint de metadados (teste de
  egress).
- [ ] Testes automatizados cobrem os casos de redirect.

--- FIM ISSUE 4 ---
```

Issue 5 — [Segurança] Adicionar CSP e cabeçalhos de segurança no nginx

```
--- ISSUE 5 ---
Título: [Segurança] Adicionar CSP e cabeçalhos de segurança no nginx
Labels sugeridas: security, severity:media

## Problema
O nginx.conf define apenas cache e gzip. Faltam Content-Security-Policy, X-Content-Type-Options,
frame-ancestors/X-Frame-Options, Referrer-Policy, Permissions-Policy e HSTS. A sessão do Supabase
fica no localStorage; sem CSP, qualquer XSS futuro roubaria o token.

## Evidência
Arquivo: nginx.conf:17-31 (nenhum add_header de segurança)
index.html:25-29 (Google Fonts externo)

## Impacto
Sem defesa em profundidade contra XSS e clickjacking; roubo de sessão e acesso total ao acervo se
houver falha futura.

## Sugestão de correção
- Adicionar CSP: default-src 'self'; script-src 'self'; connect-src 'self'
  https://<projeto>.supabase.co https://<webhook-n8n>; style-src 'self'
  https://fonts.googleapis.com; font-src https://fonts.gstatic.com; img-src 'self' data:; object-src
  'none'; base-uri 'self'; frame-ancestors 'none'.
- Adicionar X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin,
  Permissions-Policy (microphone=(self)) e HSTS.
- Repetir os add_header nos blocos location que definem Cache-Control (nginx não herda quando o
  bloco tem add_header próprio).
- Considerar hospedar a fonte localmente para fechar a CSP.

## Critérios de aceite
- [ ] curl -I no site em produção mostra todos os cabeçalhos acima, inclusive em arquivos estáticos.
- [ ] O app (login, captura por voz, chat, grafo) funciona sem violações de CSP no console.
- [ ] Tentativa de embutir o site em iframe de outro domínio é bloqueada.

--- FIM ISSUE 5 ---
```

Issue 6 — [Segurança] Endurecer o guard SSRF das Edge Functions (rebinding e AAAA)

```
--- ISSUE 6 ---
Título: [Segurança] Endurecer o guard SSRF das Edge Functions (rebinding e AAAA)
Labels sugeridas: security, severity:baixa

## Problema
O DNS é resolvido por DoH, mas o fetch usa o resolvedor do runtime (janela de rebinding, já
registrada como risco aceito). Falhas na consulta AAAA são ignoradas e a checagem passa a cobrir só
IPv4, embora o fetch possa usar IPv6.

## Evidência
Arquivo: supabase/functions/_shared/runtime.ts:90
```
const [v4, v6] = await Promise.all([query('A'),
  query('AAAA').catch(() => [] as string[])]);
```

## Impacto
Possível acesso a IP interno via rebinding ou AAAA privado em condições específicas.

## Sugestão de correção
- Tratar falha da consulta AAAA como erro (bloquear) em vez de lista vazia.
- Se o runtime permitir, conectar diretamente ao IP validado (com cabeçalho Host/SNI do domínio).
- Atualizar supabase/README.md se o risco aceito mudar.

## Critérios de aceite
- [ ] Falha simulada de AAAA resulta em 'Domínio não resolvido' e não em liberação.
- [ ] Há teste unitário para esse caso em tests/unit/ssrf.test.mjs.

--- FIM ISSUE 6 ---
```

Issue 7 — [Segurança] Escapar entradas na exportação .md e nos filtros PostgREST

```
--- ISSUE 7 ---
Título: [Segurança] Escapar entradas na exportação .md e nos filtros PostgREST
Labels sugeridas: security, severity:baixa

## Problema
generateMarkdownForObsidian grava título, URL e raw_content sem tratar quebras de linha, aspas no
campo url ou blocos de código executáveis por plugins do Obsidian. O conteúdo vem de páginas
externas via IA. Separadamente, os filtros de busca montados por string não removem aspas duplas e
geram erro 400.

## Evidência
Arquivo: src/shared/utils/formatters.js:76,80,107; src/shared/services/supabaseClient.js:49,228
```
url: "${item.source_url || ''}"
${item.raw_content || 'Sem transcrição disponível.'}
```

## Impacto
Execução de código no Obsidian do dono (plugins com JS) ou adulteração de metadados; erro de busca
com aspas.

## Sugestão de correção
- Serializar o frontmatter com escape YAML completo (JSON.stringify do valor) para title, url e
  tags.
- Neutralizar crases triplas com linguagem executável (dataviewjs, js) em raw_content, ou envolver o
  conteúdo em um bloco de citação/código seguro.
- Incluir aspas duplas e aspas simples na lista de caracteres removidos nos filtros .or() ou usar
  filtros por coluna.

## Critérios de aceite
- [ ] Título com quebra de linha ou aspas não gera chaves YAML extras.
- [ ] Conteúdo com ```dataviewjs é exportado sem ser executável.
- [ ] Busca com aspas não gera erro 400.
- [ ] Testes unitários cobrem os três casos.

--- FIM ISSUE 7 ---
```

Issue 8 — [Segurança] Mostrar origem e exigir confirmação ao pré-preencher captura por URL/share-target

```
--- ISSUE 8 ---
Título: [Segurança] Mostrar origem e exigir confirmação ao pré-preencher captura por URL/share-
target
Labels sugeridas: security, severity:baixa

## Problema
Links com ?url=, ?text= ou ?shared_url= e POSTs para /share-target abrem a janela de captura com
dados do atacante. Não há envio automático, mas nada indica ao dono que o conteúdo veio de fora.
Isso ajuda a induzir o salvamento de URLs internas (ver SSRF do n8n).

## Evidência
Arquivo: src/main.js:484; public/sw.js:35-68
```
const sharedUrl = urlParams.get('shared_url') || urlParams.get('url');
```

## Impacto
Engenharia social para capturar URL interna ou conteúdo malicioso.

## Sugestão de correção
- Aceitar apenas shared_* (remover o fallback para ?url= e ?text=) e exibir faixa 'Conteúdo recebido
  de fora, confira antes de salvar'.
- No service worker, aceitar o POST de share-target apenas com o cabeçalho de origem esperado e
  limitar tamanho/tipo do arquivo.
- Aplicar validação de destino (issue de SSRF) antes de qualquer busca.

## Critérios de aceite
- [ ] ?url= e ?text= comuns não abrem a janela de captura.
- [ ] A janela pré-preenchida mostra aviso de origem externa.
- [ ] O compartilhamento nativo do celular continua funcionando.

--- FIM ISSUE 8 ---
```

Issue 9 — [Segurança] Limitar abuso do webhook do n8n e remover localhost do CORS de produção

```
--- ISSUE 9 ---
Título: [Segurança] Limitar abuso do webhook do n8n e remover localhost do CORS de produção
Labels sugeridas: security, severity:baixa

## Problema
O webhook é público (a URL está no bundle) e só autentica após receber e parsear o corpo; cada
chamada dispara uma RPC ao Supabase. Origens http://localhost:5173/4173 estão liberadas em produção
no n8n e nas Edge Functions.

## Evidência
Arquivo: n8n/second_brain_workflow.json:10; supabase/functions/_shared/runtime.ts:13
```
"allowedOrigins": "https://2cerebro.scaladigital.com.br,http://localhost:5173"
```

## Impacto
Consumo de recursos e superfície desnecessária; CORS não substitui autenticação.

## Sugestão de correção
- Aplicar rate limiting no proxy do n8n (por IP) e limitar o tamanho do corpo antes do parse.
- Rejeitar cedo requisições sem header Authorization.
- Remover localhost de ALLOWED_ORIGINS em produção (definir a variável nas secrets) e manter
  DEFAULT_ORIGINS só para desenvolvimento.

## Critérios de aceite
- [ ] Requisições sem Authorization recebem 401 sem tocar o Supabase.
- [ ] Excesso de requisições por IP recebe 429.
- [ ] Origem http://localhost:5173 não recebe Access-Control-Allow-Origin em produção.

--- FIM ISSUE 9 ---
```

Issue 10 — [Segurança] Build e imagens: pinar yt-dlp, usar npm ci e completar .dockerignore

```
--- ISSUE 10 ---
Título: [Segurança] Build e imagens: pinar yt-dlp, usar npm ci e completar .dockerignore
Labels sugeridas: security, severity:baixa

## Problema
O media-fetcher atualiza o yt-dlp para a versão mais recente a cada start, o Dockerfile principal
usa npm install e copia o contexto inteiro, e o .dockerignore não exclui .env*, *.key,
coolify_private_key.txt, tests e .playwright-mcp. Há ainda artefatos (.playwright-mcp, __pycache__)
rastreados pelo git.

## Evidência
Arquivo: media-fetcher/Dockerfile:22; Dockerfile:8,15; .dockerignore:1-6
```
CMD ["sh", "-c", "pip install --quiet -U 'yt-dlp[default]' || true; exec python /app/server.py"]
RUN npm install
```

## Impacto
Código não revisado rodando em rede interna e risco de segredos locais entrarem em camadas de
imagem.

## Sugestão de correção
- Pinar a versão do yt-dlp (e atualizar por PR com revisão) e rodar o container com egress restrito.
- Usar npm ci no Dockerfile e commitar o package-lock.json (já existe).
- Completar o .dockerignore: .env*, *.key, *.pem, coolify_private_key.txt, tests, .playwright-mcp,
  __pycache__.
- Remover .playwright-mcp/ e media-fetcher/__pycache__/ do índice (git rm --cached) e ignorá-los no
  .gitignore.

## Critérios de aceite
- [ ] O build é reprodutível (mesma versão de yt-dlp e npm ci).
- [ ] docker build com os arquivos sensíveis na pasta não os inclui em nenhuma camada.
- [ ] git ls-files não lista .playwright-mcp nem .pyc.

--- FIM ISSUE 10 ---
```

Issue 11 — [Segurança] Versionar e testar o RLS e endurecer a autenticação

```
--- ISSUE 11 ---
Título: [Segurança] Versionar e testar o RLS e endurecer a autenticação
Labels sugeridas: security, severity:baixa

## Problema
O esquema base (brain_items, brain_chunks, match_brain_chunks) e as políticas rodadas manualmente em
auth_rls.sql não são versionados como migrations, e os testes simulam is_owner() como 'select true',
então nenhum teste prova que um não-dono é barrado. Além disso, o login é de fator único e o
bloqueio de novos cadastros é um passo manual não verificável.

## Evidência
Arquivo: supabase/auth_rls.sql:6-21; tests/helpers/db.mjs:39
```
create function public.is_owner() returns boolean language sql stable as 'select true';
```

## Impacto
Se o RLS faltar em alguma tabela ou função em produção, o anon key público daria acesso ao acervo;
credencial vazada daria acesso total.

## Sugestão de correção
- Criar migration com o esquema base e o RLS (usando um UUID do dono lido de configuração, não
  hardcoded).
- Adicionar testes de RLS (PGlite ou pgTAP) com dois papéis: dono permitido, outro usuário e anon
  negados em todas as tabelas e RPCs.
- Fixar 'set search_path = public' em is_owner() e owner_check().
- Conferir match_brain_chunks em produção (não SECURITY DEFINER) e versioná-la ou removê-la se não
  for usada.
- Desativar cadastros no Supabase Auth, ativar MFA (TOTP) para o dono e pinar verify_jwt em
  supabase/config.toml.

## Critérios de aceite
- [ ] Os testes de RLS falham se alguma política for removida.
- [ ] Consulta a pg_policies e pg_class.relrowsecurity em produção confirma RLS em todas as tabelas
  public.brain_*.
- [ ] Cadastro por novo usuário retorna erro.
- [ ] O login do dono exige segundo fator.

--- FIM ISSUE 11 ---
```